Morten Dall

Contents

Welcome . . . 7
Target audiences . . . 7
Re-use and licensing . . . 7
Contributing . . . 7
How the website is made . . . 7

I Overview

1 Syllabus . . . 11
1.1 Learning outcome and objectives . . . 11

2 Is this for you? . . . 13

3 Schedule . . . 15
3.1 Day 1 . . . 15

4 Learning design . . . 17
4.1 Clear learning goals and a strong narrative . . . 17
4.2 “Code-alongs” as the main activity . . . 17
4.3 Retrieval practice with hands-on exercises . . . 17
4.4 Linking new information with existing knowledge with discussion activities . . . 17
4.5 Minimal use of lectures . . . 17
4.6 Frequent breaks and variation in activities . . . 17
4.7 Easily accessible material that is online, always . . . 18
4.8 A safe and supportive environment . . . 18
4.9 Using origami hats to ask for help . . . 18
4.10 Being involved in the learning process . . . 18

II Pre-workshop

5 Overview . . . 21
5.1 How to read this website . . . 21
5.2 Summary of tasks . . . 21

6 Installing the programs . . . 23
6.1 Windows . . . 23

6.2 MacOS . . . 23
6.3 Linux (Ubuntu) . . . 23

7 Installing R Packages . . . 25

8 Code of conduct . . . 27
8.1 Expected behavior . . . 27
8.2 Unacceptable behavior . . . 27
8.3 Consequences of unacceptable behavior . . . 28

9 Survey . . . 29

III Sessions

10 Introduction to workshop . . . 33
10.1 The Big Picture . . . 33

11 Shiny apps . . . 35
11.1 Learning objectives . . . 35
11.2 The server and the UI . . . 35
11.3 :speech_balloon: Discussion activity: Title . . . 35
11.4 Reading task: ~4 minutes . . . 35
11.5 :book: Adding elements to our UI . . . 36
11.6 BSlib . . . 37
11.7 Summary . . . 40

12 Golem: a structured way to build and modularize your app . . . 43
12.1 Learning objectives . . . 43
12.2 Starting a golem . . . 43
12.3 :speech_balloon: Discussion activity: 01_start.R . . . 44
12.4 :technologist: Exercise activity: DESCRIPTION . . . 45
12.5 Development and modularization . . . 45
12.6 :speech_balloon: Discussion activity: add_module . . . 45
12.7 Making our first module . . . 45
12.8 Reading activity: 03_deploy.R . . . 47
12.9 Summary . . . 48

13 Building our app . . . 49
13.1 Learning objectives . . . 49
13.2 Speeding up data handling . . . 50
13.3 Making interactive plots . . . 52
13.4 :technologist: Exercise: Add the plot to our app . . . 55
13.5 Filtering and subsetting . . . 55
13.6 :technologist: Exercise: observeEvent . . . 57

13.7 :technologist: Extra Exercise: Develop the app . . . 57
13.8 Summary . . . 57

14 What next . . . 59

Appendices . . . 61

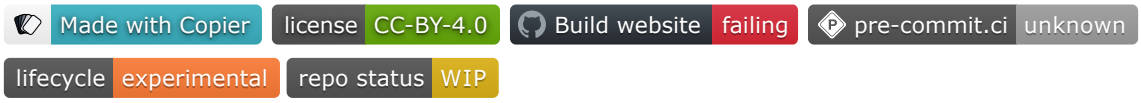
A For teachers . . . 63

References . . . 65

B Contributing . . . 67
B.1 :bug: Issues and bugs . . . 67
B.2 :pencil2: Adding or modifying content . . . 67
B.3 :file_folder: Support and infrastructure files . . . 67

C Creative Commons Attribution 4.0 International . . . 69
C.1 Creative Commons Attribution 4.0 International Public License . . . 69
C.1.1 Section 1 – Definitions. . . . 69
C.1.2 Section 2 – Scope. . . . 70
C.1.3 Section 3 – License Conditions. . . . 71
C.1.4 Section 4 – Sui Generis Database Rights. . . . 71
C.1.5 Section 5 – Disclaimer of Warranties and Limitation of Liability. . . . 72
C.1.6 Section 6 – Term and Termination. . . . 72
C.1.7 Section 7 – Other Terms and Conditions. . . . 72
C.1.8 Section 8 – Interpretation. . . . 72

Welcome



This shiny workshop is meant as a brief introduction to shiny, arrow, bslib, golem, and plotly. It is designed for doing participatory live-typing, where the teacher demonstrates the tasks on their computer connected to a projector while learners type along on their own computer. The workshop also includes reading tasks, discussion activities, and hands-on exercises.

This website contains all of the material for the workshop, including readings, exercises, presentations, live-typing material, and images. It is structured as a book, with "chapters" as sessions, in order of appearance. We make heavy use of the website throughout the workshop where "type-along" sessions almost identically follow the material on the website (with slight modifications for time or more detailed explanations).

Check out the overview section of the workshop, starting with the Syllabus.

If you plan on attending the workshop, please make sure to complete the Pre-workshop tasks to get set up and ready for the workshop. The pre-workshop tasks include a survey that you need to fill out before the workshop starts.

> Do you find this workshop material useful?
>
> If yes, please consider "starring" our GitHub repository. Starring the repository will save it to your list of saved repositories, so it's easy for you to find again later. As a plus, it helps give our project more visibility :star2:

Target audiences

This website and its content are targeted to three groups:

1. For the **learners** to use during the workshop, both to follow along and also to use as a reference after the workshop ends. A more detailed description of who the learner is can be found in Is this for you?
2. For the **teachers** to use as a guide for when they do the type-along sessions.
3. For those who are **interested in teaching**, who may not have much experience or may not know where to start, to use this website as a guide to running and teaching their own workshops.

Re-use and licensing

The workshop material is licensed under the Creative Commons Attribution 4.0 License, so the material can be used, re-used, and modified, as long as there is attribution to this source. Check out the For teachers section for more details and tips on using this material for teaching.

Contributing

Want to contribute to this workshop? Look through our CONTRIBUTING page for contribution guidelines on how to get started.

How the website is made

The workshop material is created using Quarto to write the material and create the book format, GitHub to host the Git repository of the material, and GitHub Actions with Netlify to build and host the website. The original source material for this workshop is found on the `shiny_workshop` GitHub repository.

I Overview

1 Syllabus 11
1.1 Learning outcome and objectives 11

2 Is this for you? 13

3 Schedule 15
3.1 Day 1 15

4 Learning design 17
4.1 Clear learning goals and a strong narrative 17
4.2 “Code-alongs” as the main activity 17
4.3 Retrieval practice with hands-on exercises 17
4.4 Linking new information with existing knowledge with discussion activities 17
4.5 Minimal use of lectures 17
4.6 Frequent breaks and variation in activities 17
4.7 Easily accessible material that is online, always . 18
4.8 A safe and supportive environment 18
4.9 Using origami hats to ask for help 18
4.10 Being involved in the learning process 18

1. Syllabus

TODO: Include description and overall introduction of workshop

This workshop lasts TODO: Time and is split into the following sessions, listed in the schedule, which will be covered in order:

- Introduction to the workshop
- TODO: Session title
- Concluding remarks

1.1 Learning outcome and objectives

The overall aim of this workshop is to enable you to:

1. Aim one

Broken down into specific objectives for each session, we've designed the workshop to enable you to do the following:

TODO: Session title

1. Objective one

Tangibly, during the workshop you will:

- Learn what a Shiny app is
- Create one with the golem package
- Build a dashboard using the `palmerpenguins` dataset
- Store files as parquet
- Make graphs interactive using plotly

2. Is this for you?

To help manage expectations and develop the material for this workshop, we make a few assumptions about *who you are* as a participant in the workshop.

We assume you can relate to one or more of the following:

- Have intermediate R coding skills
- Are familiar with the tidyverse framework
- Know ggplot2 syntax
- are interested in sharing your data or analysis methods through interactive application

While we have these assumptions to help focus the content of the workshop, if you have an interest in the workshop but don't fit any of the assumptions, *you are still welcome to attend*! We welcome everyone until capacity has been reached.

In addition to the assumptions above, the workshop also has a fairly focused scope, which may also help you decide if this workshop is for you:

- Get a basic introduction to making apps in R
- Learn how to make interfaces with bslib
- Build a modularized app in Shiny
- Learn about the parquet file format
- Make your graphs interactive with plotly

3. Schedule

The workshop is structured as a series of participatory live-coding sessions interspersed with hands-on exercises and group work, using either a practice dataset or some other real-world dataset. There are some lectures given, mainly at the start and end of the workshop. The general schedule outline is shown in the below table. This is not a fixed schedule of the timings of each session—some may be shorter and others may be longer. Instead, it is meant to be an approximate guide and overview.

3.1 Day 1

Time	Session topic

4. Learning design

This workshop is designed using the science of learning. Learning is when new content gets incorporated into long-term storage in the brain in a way that it can be later retrieved. Some of the best ways to enable learning is to do "active learning" by engaging with the material, practicing retrieving what was just covered, repeatedly being exposed to the content, linking the new information with what you already know, and explaining it to someone else.

We use a variety of activities to target the different "pathways" of learning. A big part of learning is being engaged in the *process* of learning, not just learning the content itself. So to help you be engaged in your own learning, we want to involve you in the reasons *why* we do the things we do during this workshop.

Throughout the workshop, we use these practices to support learning.

4.1 Clear learning goals and a strong narrative

The learning goals and the narrative are tightly connected in this workshop. Learning is easier when you have a better idea of where you are going, where you are relative to the end goal, and what to expect along the way.

4.2 "Code-alongs" as the main activity

"Code-alongs" is when the teacher types on their computer that is on the projector and explains what it means as you also type along with them. The connection between using the hands and doing something tangible helps reinforce learning. It also helps the teachers to go slower as we have to both explain and type things out. This helps you learn better. "Less is more" works very well with learning.

4.3 Retrieval practice with hands-on exercises

Hands-on exercises reinforce what was just taught through retrieval practice and repetition. It gives you a chance to engage with the material and the space to try it out on your own. This helps you form a better sense of how to do it yourself.

4.4 Linking new information with existing knowledge with discussion activities

Discussion activities encourage linking what you already know and do in your work with the new information. Because humans are social beings, we also tend to learn better when there is some social aspect to it. For example, explaining something to someone else is one of the best ways to learn, as nicely expressed in the quote "while we teach, we learn" from the Roman philosopher Seneca.

4.5 Minimal use of lectures

Lectures, while extremely common, are forms of information sharing with the lowest rates of learning. In this workshop, we use reading tasks instead, so you can go at your pace, re-read things you may not understand, and take notes, all of which support learning. To help reinforce what you just read, the teacher will briefly summarise and emphasise key points after reading tasks, to repeat the main points and target another part of the brain (the auditory pathway).

4.6 Frequent breaks and variation in activities

Breaks are how your brain can recharge and refocus. The brain is only able to effectively focus on one thing at a time in 30-50 minute blocks of time. That's why we mix together code-alongs, exercises, discussions,

and reading to give your brain some variation. Having an explicit break, even 5 minutes, is enough for the brain to better process and store what was covered.

4.7 Easily accessible material that is online, always

All the material is online on this website, so that it is easy for you to go back to after the workshop. This builds the repetition and retrieval practice part of learning. Our brains also get easily distracted, which is completely normal. So if you get distracted during the workshop, you can simply go to the material and get caught up with where we are. We also make sure to follow the material on the website closely, so that it is easier for you to follow along.

4.8 A safe and supportive environment

Mental safety in a supportive environment is critical for effective learning. It puts the brain into a more relaxed state that allows for optimal processing and ultimately learning. That's why we try to explicitly create that space, for example with the use of our Code of Conduct.

4.9 Using origami hats to ask for help

Asking for help can feel difficult, especially when you are learning and are unsure about something. To make it easier and feel safer, we use origami hats (or stickies) to show how everyone is doing. If you need help, you put up the "help" hat (usually orange) and a helper will come to you as soon as possible. If not, you can put up a "all good" hat (usually blue) to let us know you are following along. Since we often have several helpers, you usually won't wait long. This simple system lets you ask for help without interrupting the teacher or drawing attention, so you can keep following along while you wait. Our goal is to make getting help easier and more comfortable.

4.10 Being involved in the learning process

Now you know a bit more about the thoughts behind how we've designed this workshop to really try to support and enable your own learning! :tada: :grin:

II Pre-workshop

5 Overview 21
5.1 How to read this website 21
5.2 Summary of tasks 21

6 Installing the programs 23
6.1 Windows 23
6.2 MacOS 23
6.3 Linux (Ubuntu) 23

7 Installing R Packages 25

8 Code of conduct 27
8.1 Expected behavior 27
8.2 Unacceptable behavior 27
8.3 Consequences of unacceptable behavior 28

9 Survey 29

5. Overview

In order to participate in this workshop, you must complete everything in this section and finish with **completing the survey (Chapter 9.)** at the end. These tasks are meant to provide everyone with the necessary setup and a shared foundation for the workshop.

Depending on your prior knowledge and skills, these tasks could take between **TIME hrs to finish**, so we suggest planning some time to complete them. While many of the tasks are mostly readings, they are important in making sure you get the most out of the workshop.

5.1 How to read this website

We try to be consistent with the way to present certain information in this website. Specifically, we follow some conventions that you should be aware of:

- Folder names always end with `/`, for example `data/` means the data folder.
- File names always end with their file extension, for example `content.md` means the file is a Markdown file.
- R variables are always shown as is. For instance, for the code `x <- 10`, `x` is a variable because it was assigned with 10.
- Functions always end with `()`, for instance `mean()` or `read_csv()`.
- Sometimes functions have their package name appended with `::` so that you run the code from the specific package, since we likely haven't loaded the package with `library()`. For instance, to install packages from GitHub using the `{pak}` package we use `pak::pak("user/package_name")`. You'll learn about this more later.

5.2 Summary of tasks

Here's a quick overview of the tasks you need to do. Specific details about them are found as you work through the tasks.

If you haven't already, go through the workshop overview section, including the Syllabus, Is this for you?, and Learning design to get an overview of what this workshop contains and the learning design behind the workshop material.

After that, please go through the the pre-workshop tasks designed to establish a shared foundation for the start of the workshop. This includes:

1. Install a version of R, RStudio, Quarto, and Git that is as updated as possible in Chapter 6.. For some people, depending on their institution, this task can take the **longest amount of time** because you have to contact your IT to install these packages.
2. Install necessary R packages.
3. TODO: Add more tasks
4. Read the syllabus (Chapter 1.).
5. Read the Code of Conduct (Chapter 8.).
6. Complete the pre-workshop survey (Chapter 9.): This survey is pretty quick, maybe ~10 minutes. There will be some bits from the tasks that you will need to paste into this survey.

Check each section for exact details on completing these tasks.

> Tip
>
> You will be taken through the pre-workshop tasks in order by clicking the arrow at the bottom of each page :arrow_right:

6. Installing the programs

In order to participate in the workshop, you'll need to install these programs. You may already have some of them installed and if you did, please make sure that they are at least the minimum versions listed below. If not, you will need to update them.

6.1 Windows

1. R: Any version above . If you have used R before, you can confirm the version by running `R.version.string` in the Console.
2. RStudio: Any version above . If you have installed it before, check the current version by going to the menu "Help -> About RStudio".
3. Git: Select the "Click here to download" link. Git is used throughout many sessions in the workshop. When installing, it will ask for a selecting a "Text Editor" and while we won't be using this in the workshop, Git needs to know this information so choose Notepad.

6.2 MacOS

1. R: Any version above . If you have used R before, you can confirm the version by running `R.version.string` in the Console. If you use Homebrew (which we strongly recommend installing and using), installing R is as easy as opening a Terminal and running:

```
brew install --cask r
```

2. RStudio: Any version above . If you have installed it before, check the current version by going to the menu "Help -> About RStudio". With Homebrew:

```
brew install --cask rstudio
```

3. Git: Git is used throughout many sessions in the workshop. With Homebrew:

```
brew install git
```

6.3 Linux (Ubuntu)

1. R: Any version above . If you have used R before, you can confirm the version by running `R.version.string` in the Console.

```
sudo apt -y install r-base
```

2. RStudio: Any version above . If you have installed it before, check the current version by going to the menu "Help -> About RStudio".
3. Git: Git is used throughout many sessions in the workshop.

```
sudo apt install git
```

All these programs are required for the workshop, even Git. Git, which is a software program to formally manage versions of files, is used because of it's popularity and the amount of documentation available for it. Check out the online book Happy Git with R, especially the "Why Git" section, for an understanding

on why we are teaching Git. Windows users tend to have more trouble with installing Git than macOS or Linux users. See the section on Installing Git for Windows for help.

> **i** Note
>
> Some pictures may show a Git pane in RStudio, but you may not see it. If you haven't created or opened an RStudio R Project (which is taught in the introductory workshop), the Git pane *does not* show up. It only shows up in R Projects that use Git to track file changes.

> **i** Note
>
> *A note to those who have or use work laptops with restrictive administrative privileges*: You may encounter problems installing software due to administrative reasons (e.g. you don't have permission to install things). Even if you have issues installing or updating the latest version of R or RStudio, you will likely be able to continue with the workshop as long as you have the minimum version listed above for R and for RStudio. If you have versions of R and RStudio that are older than that, you may need to ask your IT department to update your software if you can't do this yourself. Unfortunately, Git is not a commonly used software for some organizations, so you may not have it installed and you *will* need to ask IT to install it. We *require* it for the workshop, so please make sure to give IT enough time to be able to install it for you prior to the workshop.

Once R, RStudio, and Git have been installed, open RStudio. *If you encounter any troubles during these pre-workshop tasks*, try as best as you can to complete the task and then let us know about the issues in the pre-workshop survey. If you continue having problems, indicate on the survey that you need help and we can try to book a quick video call to fix the problem. Otherwise, you can come to the workshop 15-20 minutes earlier to get help.

If you're unable to complete the setup procedure due to unfixable technical issues, you can use Posit Cloud (to use RStudio on the cloud) as a final solution in order to participate in the workshop. For help setting up Posit Cloud for this workshop, refer to the Posit Cloud setup guide.

7. Installing R Packages

We will be using specific R packages for the workshop, so you will need to install them. To make it easier, we've created a custom R package called `r3` that will install all the necessary packages for you for this workshop. Run these two functions in the Console by copying and pasting them to install our custom `r3` package as well as the `{tidyverse}` package.

```
install.packages("r3", repos = c("https://rostools.r-universe.dev", "https://cloud.r-project.org"))
install.packages("tidyverse")
```

Tip

Installing packages is tricky, especially on some work laptops, and doesn't always work as expected. Here are some other methods to try if you have trouble installing the packages with the code above.

For Windows users, you may have to install RTools to use either the `{pak}` or `{remotes}` method. The version of RTools you install must match the version of R that you have installed. For example, if you have R version 4.4, you need to install RTools 4.4.

First, try installing the `r3` package directly from GitHub with `{pak}`:

```
install.packages("pak")
pak::pak("rostools/r3")
```

If that doesn't work, try with `{remotes}`:

```
install.packages("remotes")
remotes::install_github("rostools/r3")
```

If you encounter any errors with either of these functions, try to restart R by going to "Sessions -> Restart R" and re-run them again. If that also doesn't work, try to complete the other tasks as best you can, complete the survey, and let us know you have a problem in the survey. We'll email you at least a week before the workshop starts to try to help you get it working.

Tip

When you see a command like `something::something()`, for example with `pak::pak()`, you would "read" this as:

> R, can you please use the `pak()` function from the `{pak}` package.

The normal way of doing this would be to load the package with `library(pak)` and then running the command (`pak()`). But by using the `::`, we tell R to directly use a function from a package, without needing to load the package and all of its other functions too. We use this trick because we only want to use the `pak()` command from the `{pak}` package and not have to load all the other functions as well. In this workshop we will be using `::` often.

8. Code of conduct

Learning happens when we feel safe and supported. Since we want you to learn during this workshop, we want you to feel safe and supported. We also want all learners to have an enjoyable and fulfilling experience while participating in this workshop. So we expect all learners to show respect, kindness, and courtesy to other learners throughout the whole workshop and to help create a positive and safe learning environment.

While most people and (sub-)cultures have an implicit, assumed set of practices and expectations for behaviour, we believe strongly in effective and clear communication. This includes making what is normally implicit expectations to instead be explicit expectations.

Teachers and helpers will uphold and embody this code in all spaces and activities of the workshop, and we expect you as the learner to do the same. In fact, by participating in this workshop, you indicate that you agree to follow these explicit expectations. We have this code of conduct to communicate to you that we really value and want to nurture a space that encourages and enhances learning. Otherwise, why would we have a workshop if you can't or aren't able to effectively learn anything from it! :zany_face:

8.1 Expected behavior

We want a welcoming, supportive, safe, and fun environment not only for ourselves as teachers and helpers, but also for all learners, regardless of background and identity. This means that we do not tolerate behaviour that is disrespectful to any of our teachers, helpers, or learners (see below for unacceptable behaviour).

We encourage the following kinds of behaviours in all interactions during this workshop that will foster a positive and professional learning environment:

- Show kindness, courtesy, and respect towards other workshop members.
- All interactions should be professional regardless of platform: either online or in-person.
- Be aware of your words by using welcoming and inclusive language. Words matter, a lot!
- Be respectful to different viewpoints and experiences.
- Gracefully accept constructive criticism.
- Focus on what is best for everyone in the workshop.

8.2 Unacceptable behavior

While we assume you will not do these behaviours, we want to clearly communicate what unacceptable behaviour is. It is any behaviour, as actions or words, that excludes, intimidates, or causes discomfort to others. We do not tolerate discrimination or harassment based on characteristics that include, though are not limited to, gender identity and expression, sexual orientation, disability, physical appearance, body size, citizenship, nationality, ethnic or social origin, pregnancy, familial status, veteran status, genetic information, religion or belief (or lack thereof), membership of a national minority, property, age, education, socio-economic status, technical choices, and experience level. We of course also don't tolerate verbally or physically violent behaviour towards others.

Examples of unacceptable behavior includes:

- Written or verbal comments that exclude people on the basis of membership of any specific group.
- The display of sexual or violent images.
- Unwelcome sexual attention, advances, and/or physical contact.
- Sustained disruption of workshop sessions or communications.
- Insults or put downs.
- Sexist, racist, homophobic, transphobic, ableist, or exclusionary jokes.

8.3 Consequences of unacceptable behavior

Learners who are asked, either in person or online, to stop any inappropriate behavior are expected to comply immediately. If a learners engages in behaviour that violates this code of conduct, the organizers, teachers, or helpers may take any action they deem appropriate, including warning the offender or removing them from the workshop.

If you feel in any way unsafe or mistreated, you may contact us privately—all communication is confidential.

Thank you for helping make this a welcoming, friendly workshop for all :smile:

> **i** Note
>
> *Acknowledgements*: Large portions of this code of conduct were taken and modified from other codes of conduct, specifically Software Carpentry and UofTCoders.

9. Survey

We have a Code of Conduct. If you haven't read it, **read it now**. The survey involves a question about Conduct. We want to make sure this workshop is a supportive and safe environment for learning, so this Code of Conduct is quite important.

If you haven't read the sections under "Workshop overview" and "Pre-workshop tasks", **please read them now**.

You're almost done. Please fill out the pre-workshop survey to finish this section:

Pre-workshop survey :tada:

See you at the workshop! :wave:

III Sessions

10 Introduction to workshop 33
10.1 The Big Picture 33

11 Shiny apps 35
11.1 Learning objectives 35
11.2 The server and the UI 35
11.3 :speech_balloon: Discussion activity: Title 35
11.4 Reading task: ~4 minutes 35
11.5 :book: Adding elements to our UI 36
11.6 BSlib 37
11.7 Summary 40

12 Golem: a structured way to build and modularize your app 43
12.1 Learning objectives 43
12.2 Starting a golem 43
12.3 :speech_balloon: Discussion activity: 01_start.R . 44
12.4 :technologist: Exercise activity: DESCRIPTION . 45
12.5 Development and modularization 45
12.6 :speech_balloon: Discussion activity: add_module 45
12.7 Making our first module 45
12.8 Reading activity: 03_deploy.R 47
12.9 Summary 48

13 Building our app 49
13.1 Learning objectives 49
13.2 Speeding up data handling 50
13.3 Making interactive plots 52
13.4 :technologist: Exercise: Add the plot to our app . 55
13.5 Filtering and subsetting 55
13.6 :technologist: Exercise: observeEvent 57
13.7 :technologist: Extra Exercise: Develop the app . 57
13.8 Summary 57

14 What next 59

10. Introduction to workshop

Introduction slides

> **i** Teacher note
>
> The slides contain speaking notes that you can view by pressing 'S' on the keyboard.

10.1 The Big Picture

TODO: Add text highlighting the bigger picture of why we teach this.

11. Shiny apps

To build a shiny app we must first understand how it works. A shiny app consists of two core components: a server and a user interface (UI). In this section, we will explain how they are connected and work together to create the shiny app.

11.1 Learning objectives

The learning objectives for this session are:

1. Understand how a shinyapp works
2. Introduce the ui and server terms
3. Introduce the package `bslib` and show how to use it to build a UI

11.2 The server and the UI

A shiny app can take a static dataset and make it available for the user to explore. Depending on how you build the app, you can allow the user to search for specific search terms, compare expression levels, or explore interactive figures to find new genes affected in your dataset.

11.3 :speech_balloon: Discussion activity: Title

Time: \3 minutes.

Discuss with the person next to you - what kind of data do you have that could benefit from being interactive? Do you have an idea for an app you want to make?

11.4 Reading task: ~4 minutes

A shiny app can get incredibly complex and consist of many interconnected scripts. But at it's simplest, a shiny app consists of two elements that work together.

At it's most simple, a shiny app can be one simple function. The following code will create a shiny app (although an empty one):

```
app <- shiny::shinyApp(
  ui = shiny::fluidPage(),
  server = function(input, output){}
)
shiny::runApp(app)
```

The UI (short for user interface) is what we see on the screen. The code contains information about what is shown on the page (such as plots, buttons and images) and in what layout (organized in columns, with a navigation bar, etc.).

The server contains information about the objects in the UI, and how the app should respond to user input. It contains information about what data is displayed in the plot, what happens when you press a button, and how the app should update when specific events happen.

A major part of building a shiny app is understanding the interplay between the UI and the server.

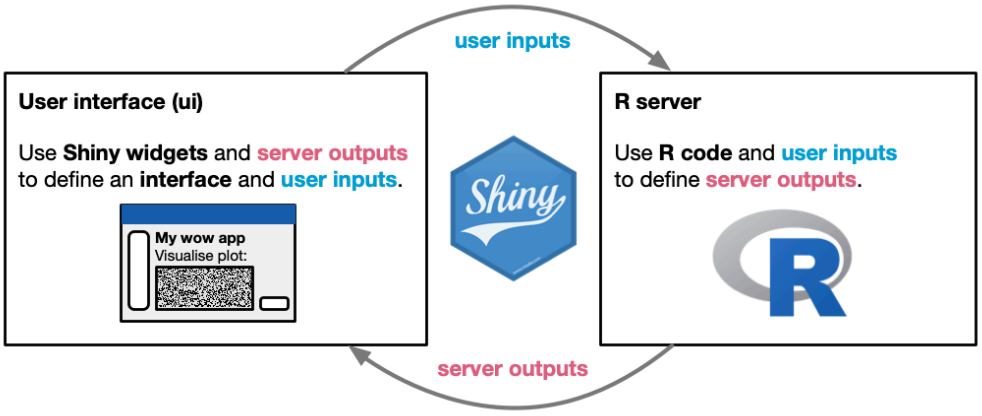


Figure 11.1: A graphical representation of the interplay between the UI and the server. From introduction to data science

11.5 :book: Adding elements to our UI

Time: \6 minutes.

Shiny comes with a lot of options for building a user interface. Often you can add plots directly into a Shiny interface. lets say we want to make a graph from the classical `iris` data set and add it into our empty app above.

```
iris_plot <- ggplot2::ggplot(iris,
                             ggplot2::aes(x = Sepal.Length, y = Petal.Length, color =
Species)) +
  ggplot2::geom_point()

iris_plot
```

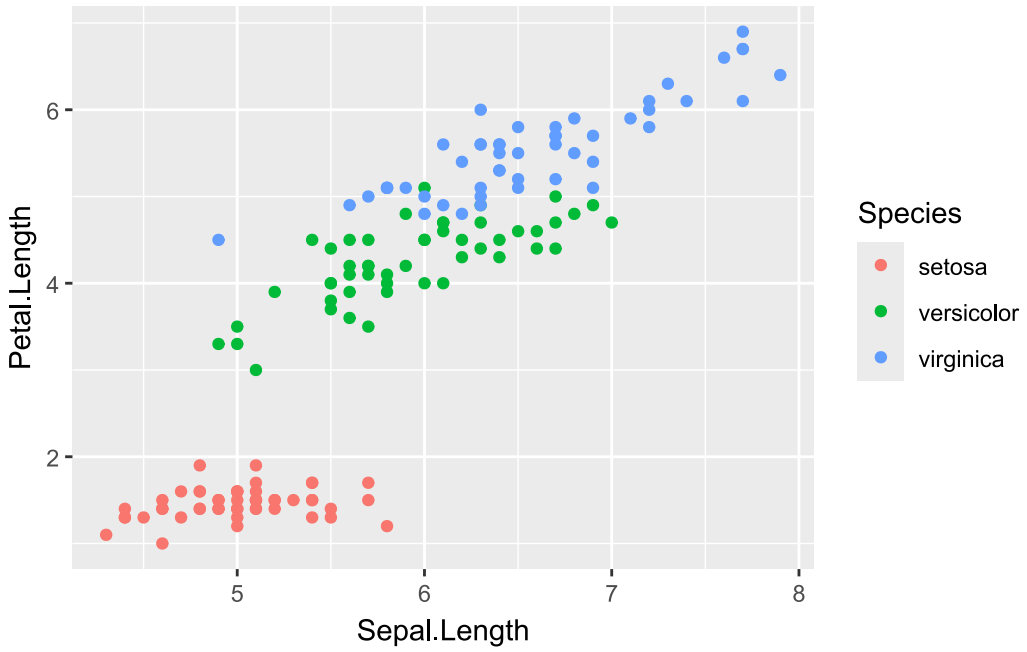


Figure 11.2: Our classical iris plot

To add this to our app, we need to tell Shiny two things: that we want a page in our page layout, and what we want the plot to contain.

```
app <- shiny::shinyApp(
  ui = shiny::fluidPage(
    shiny::plotOutput(outputId = "iris_plot"
    )
  ),
  server = function(input, output){
    output$iris_plot <- shiny::renderPlot(
      iris_plot
    )
  }
)
shiny::runApp(app)
```

If we take the code above and paste it into the console, we should get an app with the plot. We have now created content for our app!

11.6 BSlib

It is entirely possible to build a shiny app out of components from the shiny package itself. However, the "modern" shiny design focuses on a different package called "bslib" which provides access to the highly flexible bootstrap framework. Bslib allows a number of UI components with a high degree of customization, and which renders nicely, even on a small screen such as a phone.

Bslib features both pages (used to define the overall rules for our app page) and the components of the pages (especially as units of UI elements called "cards"). We can take our previous code and modify it to become a navbar page.

```
app <- shiny::shinyApp(
  ui = bslib::page_sidebar(
    title = "Our Iris plot",
    sidebar = "here is our sidebar",
    shiny::plotOutput(outputId = "iris_plot"
    )
  ),
  server = function(input, output){
    output$iris_plot <- shiny::renderPlot(
      iris_plot
    )
  }
)
shiny::runApp(app)
```

If we paste this code into our console and runs it (rerunning the code for the iris plot if necessary), we will see we now have a sidebar where we can put something in. Let's leave it empty for now, and instead focus on the other functionality of bslib we teased before - the cards.

Cards are an effective way to organize plots and other UI components. They are easy to implement - we simply wrap our UI content (in this case the Iris plot) in `bslib::card()`

```
app <- shiny::shinyApp(
  ui = bslib::page_sidebar(
    title = "Our Iris plot",
    sidebar = "here is our sidebar",
    bslib::c(shiny::plotOutput(outputId = "iris_plot")
    )
  ),
  server = function(input, output){
    output$iris_plot <- shiny::renderPlot(
      iris_plot
    )
```

```
  }
)
shiny::runApp(app)
```

In itself this does not do much. However, cards have some attributes such as headers that can be modified if we for example want to give our plot a header.

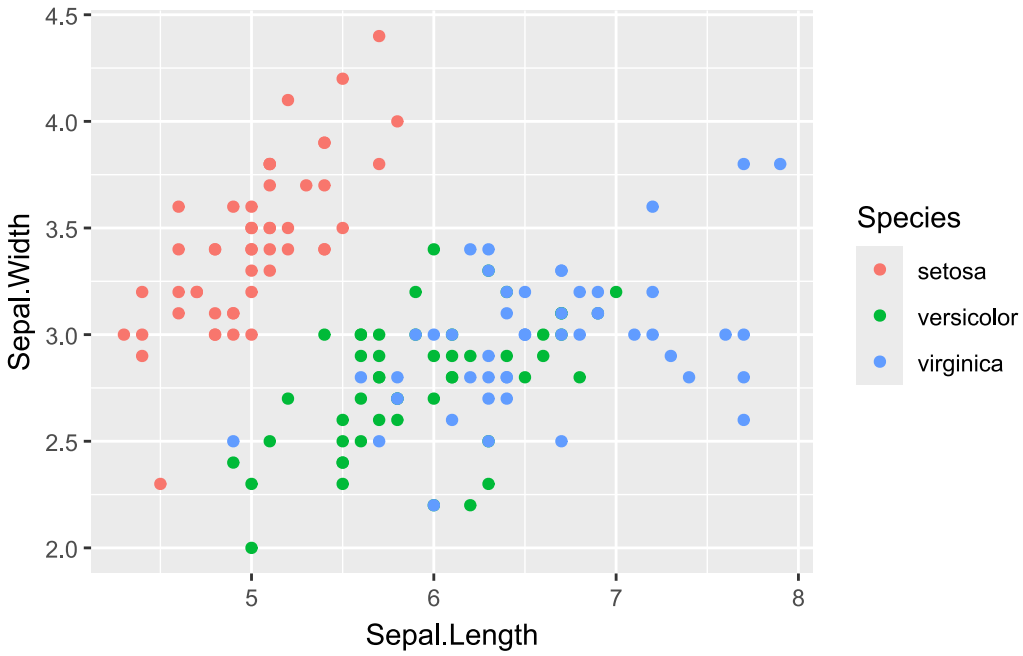
```
app <- shiny::shinyApp(
  ui = bslib::page_sidebar(
    title = "Our Iris plot",
    sidebar = "here is our sidebar",
    bslib::card(
      bslib::card_header("A beautiful plot about flowers"),
      shiny::plotOutput(outputId = "iris_plot")
    )
  ),
  server = function(input, output){
    output$iris_plot <- shiny::renderPlot(
      iris_plot
    )
  }
)
shiny::runApp(app)
```

The true power of cards becomes evident once we start having several of them. Let's make a few more plots so we can see how bslib behaves once we have multiple:

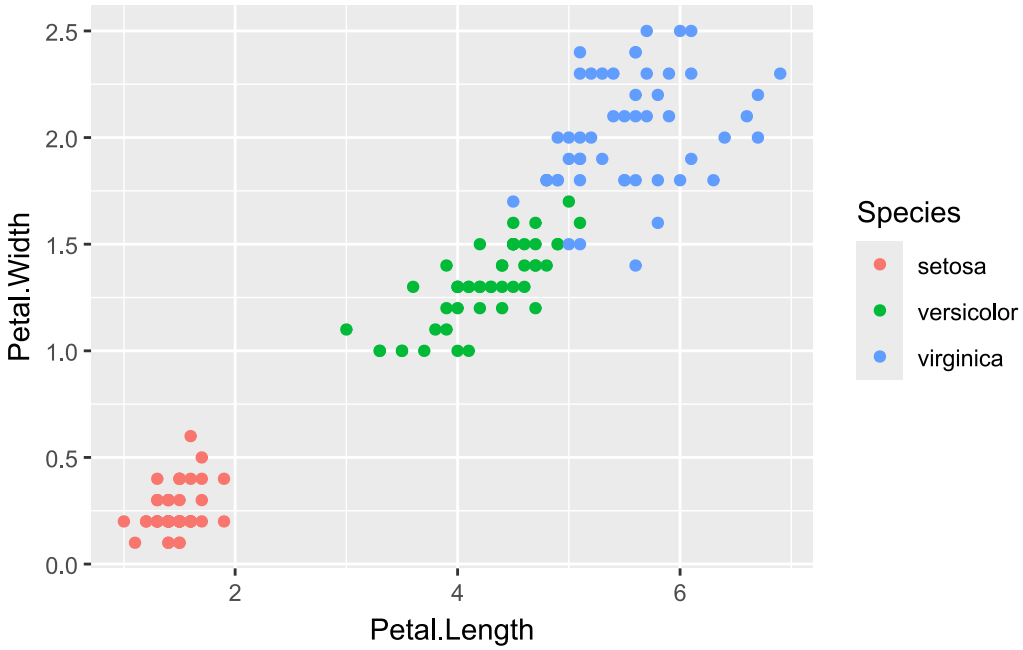
```
iris_plot_2 <- ggplot2::ggplot(iris,
                               ggplot2::aes(x = Sepal.Length, y = Sepal.Width, color
= Species)) +
  ggplot2::geom_point()


iris_plot_3 <- ggplot2::ggplot(iris,
                               ggplot2::aes(x = Petal.Length, y = Petal.Width, color
= Species)) +
  ggplot2::geom_point()

iris_plot_2
```




```
iris_plot_3
```



Lets add our two new plots to our layout, copy it to the console, and run it:

```
app <- shiny::shinyApp(
  ui = bslib::page_sidebar(
    title = "Our Iris plot",
    sidebar = "here is our sidebar",
    bslib::card(
      bslib::card_header("A beautiful plot about flowers"),
      shiny::plotOutput(outputId = "iris_plot")
    ),
    bslib::card(
      bslib::card_header("Another beautiful plot about flowers"),
      shiny::plotOutput(outputId = "iris_plot_2")
```

```
    ),
    bslib::card(
      bslib::card_header("OKay I think I've had enough flower plots for now"),
      shiny::plotOutput(outputId = "iris_plot_3")
    )
  ),
  server = function(input, output){
    output$iris_plot <- shiny::renderPlot(
      iris_plot
    )
    output$iris_plot_2 <- shiny::renderPlot(
      iris_plot_2
    )
    output$iris_plot_3 <- shiny::renderPlot(
      iris_plot_3
    )
  }
)
shiny::runApp(app)
```

It looks nice! if we want to change the order of our cards, we can add another layer from bslib, and define how many columns we want. We wrap all our cards in a function called `bslib::layout_column_wrap()` and add the argument `width  = 1/2` to get 2 columns (if this is indeed what we want). If we instead had written `width = 1/3` we would get 3 columns and so forth.

```
app <- shiny::shinyApp(
  ui = bslib::page_sidebar(
    title = "Our Iris plot",
    sidebar = "here is our sidebar",
    bslib::layout_column_wrap(
      width = 1/2,
      bslib::card(
        bslib::card_header("A beautiful plot about flowers"),
        shiny::plotOutput(outputId = "iris_plot")
      ),
      bslib::card(
        bslib::card_header("A beautiful plot about flowers"),
        shiny::plotOutput(outputId = "iris_plot_2")
      ),
      bslib::card(
        bslib::card_header("A beautiful plot about flowers"),
        shiny::plotOutput(outputId = "iris_plot_3")
      )
    )
  ),
  server = function(input, output) {
    output$iris_plot <- shiny::renderPlot(iris_plot)
    output$iris_plot_2 <- shiny::renderPlot(iris_plot_2)
    output$iris_plot_3 <- shiny::renderPlot(iris_plot_3)
  }
)
shiny::runApp(app)
```

There is much more functionality to building a good user interface and there are many resources on bslib. But for now we know how to build a Shiny interface

11.7 Summary

In this session we:

- Learned about UI and server
- Explored the relationship between outputs and render functions

- Introduced bslib and showed how it can be used to make pages and UI elements

Next session, we will show how we can build a shiny app so we have a place for our UI

12. Golem: a structured way to build and modularize your app

So far we have only run our app locally in the console. It is time to build an actual Shiny app! There are multiple ways to do this, but we will use the package called golem.

12.1 Learning objectives

The learning objectives for this session are:

1. Introduce participants to the golem module
2. Build our golem app
3. Look over golems step-by-step creation process

> **i** Reading task: Why golem? ~4 minutes
>
> There are many ways we can make a shiny app. If we want to make a very simple app and just run it locally, we can just put the code from our last exercise in a script, source it, and our shiny app will run. Nice and easy. A more structured way to make a one-script app is by making a new shiny app project. If we have shiny installed we can go to `file`, `new project` and select `shiny application`. This creates a one script app, containing the essentials. Running this app is easy, just click the `run app` button and you are flying. For many apps, this setup is perfectly adequate and you can deploy it to the internet if you have somewhere to host it.
>
> What then, if our apps are more complex? This is where a package like `golem` becomes handy. `golem` is a package focused on building a production-ready, robust shiny app. Once initiated it creates a range of files that step by step helps you establishing a lot of groundwork so you don't have to do it later. It helps you setup a github repository and it can also help you deploy the app to a number of services once you are done.
>
> One of the most useful features from `golem` is that it natively helps you modularizing your app. When you modularize you split your app into smaller components. Suggest you have an app with multiple pages, where each page provides a different range of buttons, sliders and plots. How do you keep track of which button goes where? Scripts can easily get out of hand that way. `golem` makes one script pr. page, containing info for the UI and server that goes together. So if your app contains a welcome page, a settings page, and a plot page, `golem` will make three files for you - a welcome file, a settings file and a plot file. Additionally it will make a server file and a ui file, where all your modules are tied together.
>
> Is `golem` overkill for what we are doing here? Certainly! But I want to introduce it to you already so you know it, and once you start developing more complex apps, you know where to go.

12.2 Starting a golem

If we want to make a new app and we have golem installed, we can open the menu and select `new project` (like we did above) and pick `Package for shiny app using golem`. If we want to do this from the console.

Lets make an app about something fun - penguins. We will call it "penguin app" (or "penguinapp" in R writing)

```
golem::create_golem(path = "C:/Your_R_projects_destination/penguinapp"
                    )
```

Running this command (with a package name of your choice) creates a new R project called penguinapp. We will see this has quite a few more files than our standard Shiny project.

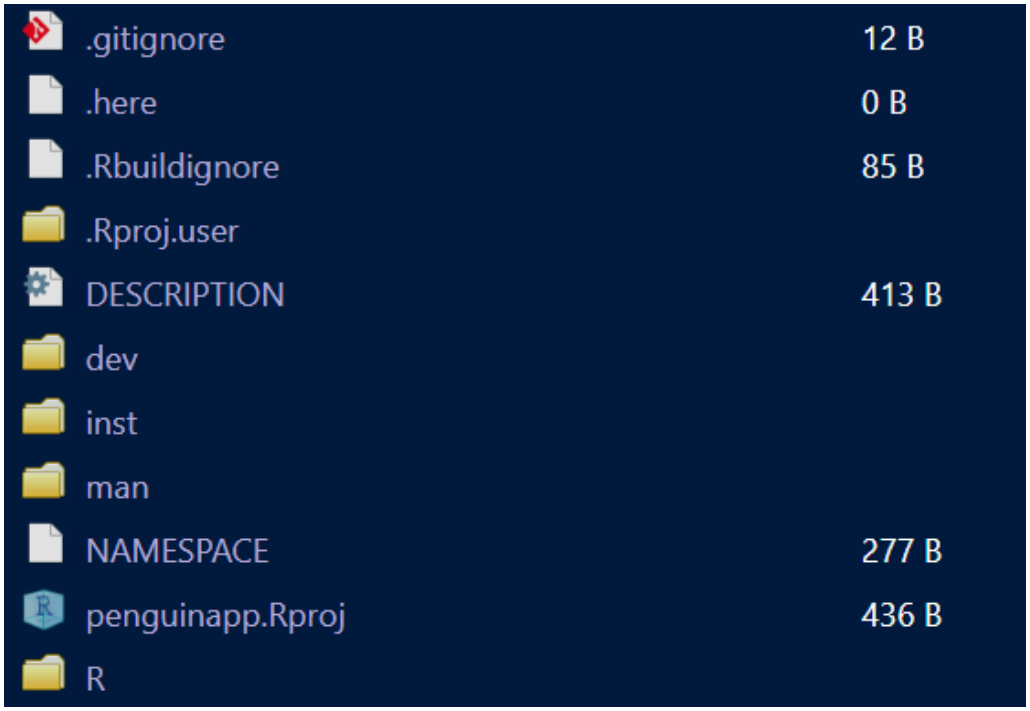


Figure 12.1: Overview of the files created by golem

This is actually a functional Shiny app, though at this stage it works a bit differently. We can run it with the command `golem::run_dev()`. Running this command will take us to a golem welcome page with resources and information. This is very similar to the example in our shiny project above.

To understand what golem really offers, it is beneficial to look at the dev folder. In this folder we will see three documents:

- 01_start.R
- 02_dev.R
- 03_deploy.R

These three files contains a step by step guide to how you build and deploy a shiny app. You don't have to do it all (or in the order you are asked to), but `golem` gives a suggested path through the development process, starting right after we create our app.

12.3 :speech_balloon: Discussion activity: 01_start.R

Time: ~5 minutes.

We don't have time to go too much into detail this session with what you do in 01_start - but take a few minutes to open the file, and discuss the content with the person next to you. What kind of information does `golem` suggest you setup? Do you recognize any terms in this file?

{{< text_snippet sticky_up >}} `golem` helps you with a lot of housekeeping you usually end up doing last minutes, including making a DESCRIPTION file (which contains a lot of input about required packages, info on the author, and whether you have coined a specific license to the usage of your package.) It also helps you with a number of other helpful tools for reproducible development such as using git and github. By doing this ahead of time, you do all the legwork before development begins, and you won't have to do it once your app has grown in size.

At this point we will limit ourselves to making a DESCRIPTION file by running the following:

```
golem::fill_desc(
  pkg_name = "penguinapp", # The name of the golem package containing the app (typically
lowercase, no underscore or periods)
  pkg_title = "PKG_TITLE", # What the Package Does (One Line, Title Case, No Period)
  pkg_description = "PKG_DESC.", # What the package does (one paragraph).
```

```
  authors = person(
    given = "AUTHOR_FIRST", # Your First Name
    family = "AUTHOR_LAST", # Your Last Name
    email = "AUTHOR@MAIL.COM", # Your email
    role = c("aut", "cre") # Your role (here author/creator)
  ),
  repo_url = NULL, # The URL of the GitHub repo (optional),
  pkg_version = "0.0.0.9000", # The version of the package containing the app
  set_options = TRUE # Set the global golem options
)
```

A DESCRIPTION file is also where you note what packages are required to run your app. This can be done by using the function `usethis::use_package(PACKAGE NAME)` Normally it is sufficient to call the package on it's own, with a notable exception being the `tidyverse` package - here you should add the following: `usethis::use_package("tidyverse", type = "suggests")` because `tidyverse` is a collection. Adding used packages to the DESCRIPTION file makes it easier to deploy the app, because it makes it clear what packages need to be installed on your remote server.

12.4 :technologist: Exercise activity: DESCRIPTION

Time: ~5 minutes.

Fill out the information for the description file and run the code in the penguinapp folder. Add relevant packages we have used (or you expect we will use) using `usethis`.Think about `tidyverse` packages - what tools from that collection do we actually use?

{{< text_snippet sticky_up >}}

12.5 Development and modularization

We will in this workshop be bad examples and start development on our penguin app before doing any of the housekeeping. if we are leading by example, this is a poor example indeed! But jokes aside, our second file 02_dev.R contains the scripts we need to build our app.

There are many different functions in this file and we do not need all of them - a lot of these functions are advanced edge cases in case you want to implement specific programs or programming languages. It does contain some helpful functions:

- usethis::use_github() helps you share your project on github
- usethis::use_data_raw() is helpful if you want to use data in your app
- golem::add_utils("helpers", with_test = TRUE) creates a script for making helper functions, in case you develop your own functions to be used in the app

The rest of the functions are beyond the scope of this workshop, except one: `golem::add_module()`

12.6 :speech_balloon: Discussion activity: add_module

Time: ~5 minutes.

Open the add_module() function. Discuss with the person next to you what it does, and why this could be useful

12.7 Making our first module

We want to make an app about the `penguins` dataset. In general, it is considered good design form to consider what the app should look like before we start coding. Lets take a look at the data and see what we can do with it:

```
dplyr::slice_head(penguins,n = 10)
```

```
   species    island bill_len bill_dep flipper_len body_mass    sex year
1   Adelie Torgersen     39.1     18.7         181      3750   male 2007
2   Adelie Torgersen     39.5     17.4         186      3800 female 2007
```

```
3   Adelie Torgersen     40.3     18.0         195      3250 female 2007
4   Adelie Torgersen       NA       NA          NA        NA   <NA> 2007
5   Adelie Torgersen     36.7     19.3         193      3450 female 2007
6   Adelie Torgersen     39.3     20.6         190      3650   male 2007
7   Adelie Torgersen     38.9     17.8         181      3625 female 2007
8   Adelie Torgersen     39.2     19.6         195      4675   male 2007
9   Adelie Torgersen     34.1     18.1         193      3475   <NA> 2007
10  Adelie Torgersen     42.0     20.2         190      4250   <NA> 2007
```

We see we have eight different columns with data about the penguins - species, island, bill length, etc. Lets make a module called "overview" where we will make a an explorative plot in the next chapter. For now, we build the module:

```
golem::add_module(name = "overview")
```

you will notice the standard golem output includes an argument stating `with_test=TRUE`. This is not necessary for our app, and truth being told I have yet to grasp the potential and utility in the tests offered by golem. Know they are there, and you can look for information if this is of interest to you.

If we run this command, we should now get a new file - we get a console output stating `File created at R/mod_overview.R`. Looking in our R folder we see there now is a script called mod_overview.R. The script contains several chunks - lets go through them in turn

```
mod_overview_ui <- function(id) {
  ns <- NS(id)
  tagList(

  )
}
```

This first chunk is the UI component. In the function called `tagList` (from shiny) we can add all the UI components we desire - cards, pages, you name it.

Next thing we see is the server:

```
mod_overview_server <- function(id){
  moduleServer(id, function(input, output, session){
    ns <- session$ns

  })
}
```

In this, we put the server component in. So far, this is very similar to our initial example. The next part is commented out (with #) but is quite interesting:

```
## To be copied in the UI
# mod_overview_ui("overview_1")

## To be copied in the server
# mod_overview_server("overview_1")
```

`golem` gives us to two functions we can paste into our UI and server script. These are also found in the /R folder and are called "app_ui" and "app_server". If we open app_ui we will see the following in the top part:

```
app_ui <- function(request) {
  tagList(
    # Leave this function for adding external resources
    golem_add_external_resources(),
    # Your application UI logic
    fluidPage(
      golem::golem_welcome_page() # Remove this line to start building your UI
    )
```

```
  )
}
```

Here we see the description of the UI - it describes the `golem` demo page we saw before, with all the golem resources. Standard is `shiny::fluidPage`, but if you want to use `bslib` as we discussed in the previous chapter, it is worth upgrading the page type to a one from this package - the equivalent is `bslib::page_fluid()`. Using the default `shiny::fluidPage` can cause some unexpected interactions with `bslib elements`.

In this case, we like the `golem` page, but we want to make a page of our own. We will turn it into a page with a navigation bar, and add our new module. We replace `fluidPage` with `bslib::page_navbar()`, and wrap the `golem_welcome_page()` in `bslib::nav_panel()`, telling shiny that we want it to be displayed as a panel in the navigation bar. We also give a title for the tab.

Then we copy the text from the golem module (`mod_overview_ui("overview_1")`) into another `nav_panel()` instance, and gives it a title.

```
app_ui <- function(request) {
  tagList(
    # Leave this function for adding external resources
    golem_add_external_resources(),
    # Your application UI logic
    bslib::page_navbar(
      bslib::nav_panel(
        title = "golem welcome page",
        golem::golem_welcome_page()
      ),
      bslib::nav_panel(
        title = "Our awesome penguin app",
        mod_overview_ui("overview_1")
      )

    )
  )
}
```

Next, we open the app_server script. We see it is currently empty - the welcome page is apparantly all UI and no server. We want to add a server of our own, and copy the server script from our module file (`mod_overview_server("overview_1")`)

```
app_server <- function(input, output, session) {
  # Your application server logic
  mod_overview_server("overview_1")
}
```

if we run `golem::run_dev()` we will now get an app with two tabs - the golem page and an empty page. We have made a modularized app! additional modules can be added the same way, and the size of scripts kept to a manageable size. Note that the background is purple, which is being hard set by golem - if we delete the golem welcome page it will go away.

12.8 Reading activity: 03_deploy.R

Time: \5 minutes.

In our dev folder we had a final file - 03_deploy.R. This one contains helper functions for when your app is ready to be shared with the world. it helps you build functions from your app into a package (and helps you check if it is good enough to be added to CRAN, where we download our packages from), and it contains a number of options to help prepare your app to be hosted on platforms such as posit connect or other platforms. Deploying is beyond the scope of this workshop, but this file will give you a number of options you can look up and find a solution that works for you. If you want an easy way to share apps, I encourage you to check out posit connect - they have free plans if your app is small, and once your account is connected, it requires few steps to upload and update apps.

{{< text_snippet sticky_up >}}

12.9 Summary

You have now learned

- How to build an app with golem
- How the dev files work
- How modularization works
- Where you can find information about how to share your app with the world

13. Building our app

We have now learned how to setup our app and modularize it so it is easier to navigate the code. Now it's time to put something in it!

13.1 Learning objectives

The learning objectives for this session are:

1. Objective one

13.2 Speeding up data handling

Reading task: ~4 minutes

Because our app is an example, speed is not really an issue when loading in data. But imagine you had a large sequencing dataset with thousands of genes from hundreds of samples - suddenly loading speed (and performance) can become an issue. If data can be organized as csv files, this is usually a good option - csv files can be loaded in rapidly using the `vroom()` function from the package of the same name. But if your dataset is very large and your app feels clunky, there are a few options you can utilize

If file size is the issue, consider converting file format to the parquet format. Parquet is similar to csv but structures data in a different way. Generally, the form is optimized for machines rather than humans, as we cannot open and interact with it the way we can with a csv. But for our apps, this does not matter too much.

You can convert data to parquet using the `arrow` package. If you want to save our penguin dataset in a folder called "data-raw", you use the command

```
#create the data-raw folder

usethis::use_data_raw()

#save our data as parquet
arrow::write_parquet(x = penguins,
                     sink = here::here("data-raw/penguins.parquet"))
```

Then data can be read in using `arrow::read_parquet("data-raw/penguins.parquet")` If you are interested in how much more efficient parquet is, this blog post provides some indication: for a data set with 11 mio rows, parquet files were 6.8x smaller than csv, and processed 22-60x faster.

If loading is not the issue, but data wrangling is (for example filtering steps) then an alternative is to use duckplyr. Duckplyr uses the powerful duckdb engine to compute results faster. `duckplyr` works as a faster drop-in for dplyr. If you call `library(duckplyr)` in a normal R script, `duckplyr` will overwrite dplyr methods. In a shiny app (especially using golem where we modularize things) it can be a bit more tricky - you should explicitely call `duckplyr::methods_overwrite()` in the `run_app.R` script just before `with_golem_options()`. I have not verified this approach yet - but you should be able to get the speed by adding `duckplyr::as_duckplyr_tibble()` in the top of your data wrangling steps:

```
penguins |>
  duckplyr::as_duckplyr_tibble() |>
  dplyr::filter(sex == "female")
```

```
The duckplyr package is configured to fall back to dplyr when it encounters an
incompatibility. Fallback events can be collected and uploaded for analysis to
guide future development. By default, data will be collected but no data will
be uploaded.
ℹ Automatic fallback uploading is not controlled and therefore disabled, see
  `?duckplyr::fallback()`.
✔ Number of reports ready for upload: 42.
→ Review with `duckplyr::fallback_review()`, upload with
  `duckplyr::fallback_upload()`.
ℹ Configure automatic uploading with `duckplyr::fallback_config()`.
```

```
Warning: `as_duckplyr_tibble()` was deprecated in duckplyr 1.0.0.
ℹ Please use `as_duckdb_tibble()` instead.
```

```
# A duckplyr data frame: 8 variables
   species island    bill_len bill_dep flipper_len body_mass sex     year
   <fct>   <fct>        <dbl>    <dbl>       <int>     <int> <fct>  <int>
 1 Adelie  Torgersen     39.5     17.4         186      3800 female  2007
 2 Adelie  Torgersen     40.3     18           195      3250 female  2007
 3 Adelie  Torgersen     36.7     19.3         193      3450 female  2007
 4 Adelie  Torgersen     38.9     17.8         181      3625 female  2007
 5 Adelie  Torgersen     39.2     19.6         195      4675 female  2007
 6 Adelie  Torgersen     34.1     18.1         193      3475 female  2007
 7 Adelie  Torgersen     42       20.2         190      4250 female  2007
 8 Adelie  Torgersen     37.8     17.1         186      3300 female  2007
 9 Adelie  Biscoe       37.8     18.3         174      3400 female  2007
10 Adelie  Biscoe       35.9     19.2         189      3800 female  2007
# ℹ more rows
```

These methods work best if you have data that can be stored as a dataframe. For some of our omics data (like single cell RNAseq data), this can be notoriously difficult and objects from for example `seurat` can take a long time to load into R. This makes it difficult to use the tools above. My best advice here is to delete as much as you can from the Seurat datasets they often end up being filled with normalizations. Consider checking out the package DietSeurat or setting aspects of the seurat object (such as counts and scale.data) to NULL.

13.3 Making interactive plots

After all this reading, it is time to code some plots. We can start by making a plot exploring the possible relation between bill length and flipper length for different species of penguins.But first, let's go in and delete the golem welcome page:

```
app_ui <- function(request) {
  tagList(
    # Leave this function for adding external resources
    golem_add_external_resources(),
    # Your application UI logic
    bslib::page_navbar(
      bslib::nav_panel(
        title = "Our awesome penguin app",
        mod_overview_ui("overview_1")
      )

    )
  )
}
```

We first build the UI code:

```
mod_overview_ui <- function(id) {
  ns <- NS(id)
  tagList(
    bslib::card(
      bslib::card_header("penguin plot"),
      shiny::plotOutput(outputId = "penguinplot")
    )
  )
}
```

Then we need to decide what should be rendered on the server side:

```
mod_overview_server <- function(id){
  moduleServer(id, function(input, output, session){
    ns <- session$ns

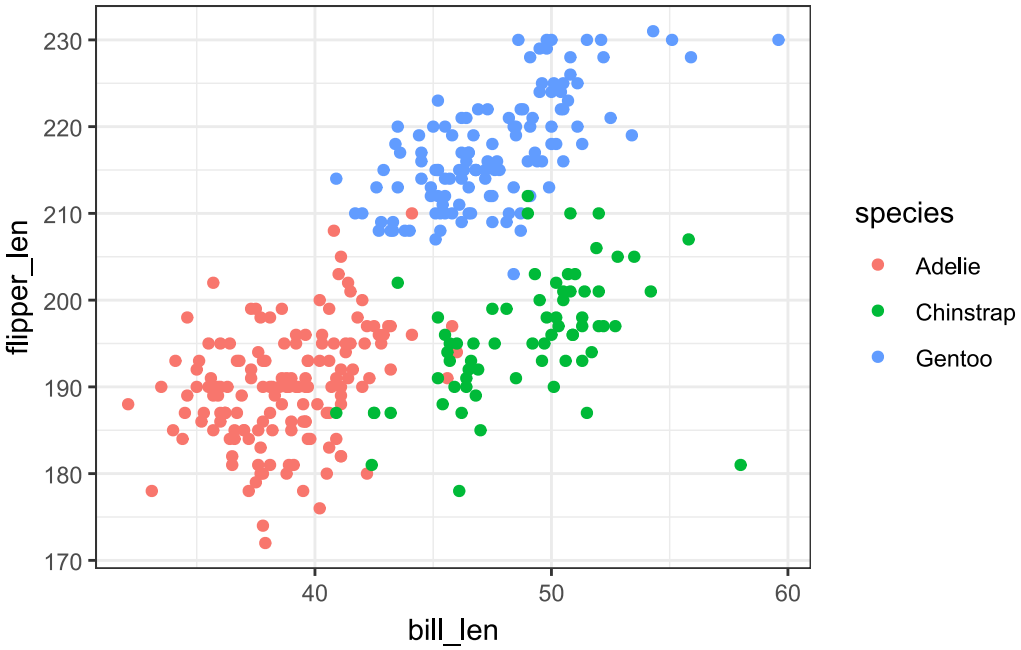
    output$penguinplot <- shiny::renderPlot({
      ggplot2::ggplot(
        penguins,
        ggplot2::aes(
          x = bill_len,
          y = flipper_len,
          color = species
        )
      )+
        ggplot2::geom_point()+
        ggplot2::theme_bw()
    })
  })
}
```

This should modify our app to render a plot like the following

```
      ggplot2::ggplot(
        penguins,
        ggplot2::aes(
          x = bill_len,
          y = flipper_len,
          color = species
        )
```

```
      )+
        ggplot2::geom_point()+
        ggplot2::theme_bw()
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```



But surprisingly it does not work! Our app is just blank? Where is our beautiful plot? To render it properly, we need to add a small wrapper to our outputId. It should be changed from `outputId = "penguinplot"` to `outputId = ns("penguinplot)"`. You may notice our server code contains a reference to ns in the line `ns <- session$ns`. What this does is essentially a way for our modularized app to navigate where input and output id's belong in our various modules. Always add `ns("")` around input and output IDs!

```
mod_overview_ui <- function(id) {
  ns <- NS(id)
  tagList(
    bslib::card(
      bslib::card_header("penguin plot"),
      shiny::plotOutput(outputId = ns("penguinplot"))
    )
  )
}
```

If we now run `golem::run_dev()` we see our plot. But right now it is static. What if we wanted to make it more interactive? There is a powerful alternative to ggplot called `plotly`. In this package interactivity is native. But the syntaxt of plotly is different. The equivalent of the above would be

```
Warning: Ignoring 2 observations
```

We see here there are several nice functionalities - we can zoom, we can export and there is a native tooltip - if we hover above individual points, we can get info about them. We can customise the content of these, but it requires some finesse. Luckily there is a way we can convert our ggplot to plotly easily, using the `ggplotly` function.

```
penguin_plotly <- ggplot2::ggplot(
        penguins,
        ggplot2::aes(
          x = bill_len,
          y = flipper_len,
          color = species
        )
      )+
        ggplot2::geom_point()+
        ggplot2::theme_bw()

plotly::ggplotly(penguin_plotly)
```

We can customize tooltip information in `ggplotly`. We can for example add information about the sex of the penguin by adding it as a text info in ggplot, and call it in `gglotly`:

```
penguin_plotly <- ggplot2::ggplot(
        penguins,
        ggplot2::aes(
          x = bill_len,
          y = flipper_len,
          color = species
        )
      )+
        ggplot2::geom_point(
```

```
          ggplot2::aes(
            text = sex
          )
        )+
        ggplot2::theme_bw()
```

```
Warning in ggplot2::geom_point(ggplot2::aes(text = sex)): Ignoring unknown
aesthetics: text
```

```
plotly::ggplotly(penguin_plotly,
                 tooltip = "text")
```

Before we try to run it in our app, we have one important thing to do in the UI - we must change our `shiny::plotOutput(outputId = ns("penguinplot"))` to `plotly::plotlyOutput(outputId = ns("penguinplot"))` and likewise change
`output$penguinplot <- shiny::renderPlot({` to `output$penguinplot <- plotly::renderPlotly({`
Once this is done, we should be good to go.

13.4 :technologist: Exercise: Add the plot to our app

Time: ~6 minutes.
Add the plot above to our app and try to run it
{{< text_snippet sticky_up >}}

13.5 Filtering and subsetting

So far, most of what we have done could be achieved in a static document. But shiny offers a lot more - particularly the ability to respond to our inputs. Lets take our plot above - say we want the ability to filter our plot based on what island the penguins are from.

```
penguins |>
  dplyr::distinct(island)
```

```
     island
1 Torgersen
2    Biscoe
3     Dream
```

We see we have three options - Torgersen, Biscoe and Dream. We can make an `input` containing these three options in our ui, plus a fourth containing all of them.

```
mod_overview_ui <- function(id) {
  ns <- NS(id)
```

```
  tagList(
    bslib::card(
      bslib::card_header("penguin plot"),
      shiny::radioButtons(inputId = ns("island"),
                          label = "Select island",
                          choices = c("all",
                                      "Torgersen",
                                      "Biscoe",
                                      "Dream"),
                          selected = "all"),
      plotly::plotlyOutput(outputId = ns("penguinplot"))
    )
  )
}
```

If we run `golem::run_dev()` we will now see a radiobutton with four choices. But they don't do anything yet. To link the choice to our plot, we must modify the code for our plot. If we just want to filter the data set based on which island has been chosen, we need the name of our `input` - in this case "island". we could just do something like the following:

```
penguin_subset <- penguins |>
  dplyr::filter(island == input$island)

penguin_plotly <- ggplot2::ggplot(
        penguin_subset,
        ggplot2::aes(
          x = bill_len,
          y = flipper_len,
          color = species
        )
      )+
        ggplot2::geom_point(
          ggplot2::aes(
            text = sex
          )
        )+
        ggplot2::theme_bw()

plotly::ggplotly(penguin_plotly,
                 tooltip = "text")
```

This will work for chosing the island, but how about if we want the option to show the whole dataset? We can do it with an if/else statement. If we have selected "all" we use the entire penguins dataset - else we will filter for the content of input$island. In R code it looks like this:

```
if(input$island == "all"){
  penguin_subset <- penguins
}
else {
  penguin_subset <- penguins |>
  dplyr::filter(island == input$island)
}

penguin_plotly <- ggplot2::ggplot(
        penguin_subset,
        ggplot2::aes(
          x = bill_len,
          y = flipper_len,
          color = species
        )
      )+
```

```
        ggplot2::geom_point(
          ggplot2::aes(
            text = sex
          )
        )+
        ggplot2::theme_bw()

plotly::ggplotly(penguin_plotly,
                 tooltip = "text")
```

We can now change our plot based on input! This is just one example of reactivity in shiny. There are many ways to change input - for a gallery check this resource

13.6 :technologist: Exercise: observeEvent

Time: ~6 minutes.

Another helpful function to know is `shiny::observeEvent()`. this function is coded into the server part and when the condition is triggered, the code is run. Try this out by adding the following:

1. In the UI, add an action button `shiny::actionButton(inputId = ns("test")`.
2. In the server add
3. Run the app, press the button and watch the console for the output

13.7 :technologist: Extra Exercise: Develop the app

if you have extra time, you can try making our penguin app nicer.Fix the axis lengths through `xlim` and`ylim` on the plot or changing the color scheme. Or try adding a different module with a different plot.

13.8 Summary

- We have now learned how to speed up loading and performance of our app
- We also know how to make plots interactive
- Finally we have learned how to make Shiny responsive and change the output based on our input.

14. What next

You have now taken your first step into a much larger world. If you have feedback or questions about Shiny, feel free to shoot an email

Appendices

A. For teachers

A more general guide to teaching or helping is found on the Guides website. This page has information relevant specifically to this workshop.

- For created teams, assign one teacher or helper to each group to help them specifically during the final group project.

References

B. Contributing

Check out the Contributing Guide for more details on how to contribute. For guidelines on how to be a helper or teacher, check out the For teachers page. As always, make sure to adhere to the Code of Conduct.

B.1 :bug: Issues and bugs

The easiest way to contribute is to report issues or bugs that you might find while using or reading the workshop material. You can do this by creating a new issue on our GitHub repository.

B.2 :pencil2: Adding or modifying content

To contribute to the workshop, you first need to install uv and justfile. We use uv and justfile to manage our project, such as to run checks and build the workshop website. Both the uv and justfile websites have a more detailed guide on using uv, but below are some simple instructions to get you started.

It's easiest to install uv first and then install justfile using uv. Then, to install justfile, run:

```
uv tool install rust-just
```

We keep all our development workflows in the `justfile`, so you can explore it to see what commands are available. To see a list of commands available, run:

```
just
```

As you contribute, make sure your changes will pass our checks by opening a terminal so that the working directory is the root of this workshop's parent/root folder and running:

```
just run-all
```

When committing changes, please try to follow Conventional Commits as Git messages. Using this convention allows us to be able to automatically create a release based on the commit message by using Cocogitto. If you don't use Conventional Commits when making a commit, we will revise the pull request title to follow that format. That's because we use squash merges when merging pull requests, so all other commits in the pull request will be squashed into one commit.

B.3 :file_folder: Support and infrastructure files

- `.copier-answers.yml`: Contains the answers you gave when copying the project from the template. **You should not modify this file directly.**
- `.gitignore`: This ignore file tells Git which files to not track. Unless you know what you are doing, it's best to not touch this file.
- `_quarto.yml`: Quarto configuration file for the website, including settings for the website, such as the theme, navigation, and other options.
- `_metadata.yml`: Quarto metadata file for the website, including information about the project, such as the titles and GitHub names.
- `.pre-commit-config.yaml`: Pre-commit configuration file for managing and running checks before each commit.
- `.config/`: Contains configuration files for various tools used in the project, such as:
  - `typos.toml`: typos spell checker configuration file.
  - `rumdl.toml` and `panache.toml`: rumdl and Panache configuration file for formatting Markdown files in the project.
- `.zenodo.json`: Structured citation metadata for your project when archived on Zenodo. This is used to add the metadata to Zenodo when a GitHub release has been uploaded to Zenodo.
- `justfile`: `just` configuration file for scripting project tasks.

- `.editorconfig`: Editor configuration file for EditorConfig to maintain consistent coding styles across different editors and IDEs.
- `CONTRIBUTING.md`: Guidelines for contributing to the project.
- `.github/`: Contains GitHub-specific files, such as issue and pull request templates, workflows, dependabot configuration, pull request templates, and a CODEOWNERS file.

C. Creative Commons Attribution 4.0 International

Creative Commons Corporation ("Creative Commons") is not a law firm and does not provide legal services or legal advice. Distribution of Creative Commons public licenses does not create a lawyer-client or other relationship. Creative Commons makes its licenses and related information available on an "as-is" basis. Creative Commons gives no warranties regarding its licenses, any material licensed under their terms and conditions, or any related information. Creative Commons disclaims all liability for damages resulting from their use to the fullest extent possible.

Using Creative Commons Public Licenses

Creative Commons public licenses provide a standard set of terms and conditions that creators and other rights holders may use to share original works of authorship and other material subject to copyright and certain other rights specified in the public license below. The following considerations are for informational purposes only, are not exhaustive, and do not form part of our licenses.

- **Considerations for licensors:** Our public licenses are intended for use by those authorized to give the public permission to use material in ways otherwise restricted by copyright and certain other rights. Our licenses are irrevocable. Licensors should read and understand the terms and conditions of the license they choose before applying it. Licensors should also secure all rights necessary before applying our licenses so that the public can reuse the material as expected. Licensors should clearly mark any material not subject to the license. This includes other CC-licensed material, or material used under an exception or limitation to copyright. More considerations for licensors.
- **Considerations for the public:** By using one of our public licenses, a licensor grants the public permission to use the licensed material under specified terms and conditions. If the licensor's permission is not necessary for any reason–for example, because of any applicable exception or limitation to copyright–then that use is not regulated by the license. Our licenses grant only permissions under copyright and certain other rights that a licensor has authority to grant. Use of the licensed material may still be restricted for other reasons, including because others have copyright or other rights in the material. A licensor may make special requests, such as asking that all changes be marked or described. Although not required by our licenses, you are encouraged to respect those requests where reasonable. More considerations for the public.

C.1 Creative Commons Attribution 4.0 International Public License

By exercising the Licensed Rights (defined below), You accept and agree to be bound by the terms and conditions of this Creative Commons Attribution 4.0 International Public License ("Public License"). To the extent this Public License may be interpreted as a contract, You are granted the Licensed Rights in consideration of Your acceptance of these terms and conditions, and the Licensor grants You such rights in consideration of benefits the Licensor receives from making the Licensed Material available under these terms and conditions.

C.1.1 Section 1 – Definitions.

a. **Adapted Material** means material subject to Copyright and Similar Rights that is derived from or based upon the Licensed Material and in which the Licensed Material is translated, altered, arranged, transformed, or otherwise modified in a manner requiring permission under the Copyright and Similar Rights held by the Licensor. For purposes of this Public License, where the Licensed Material is a musical work, performance, or sound recording, Adapted Material is always produced where the Licensed Material is synched in timed relation with a moving image.

b. **Adapter's License** means the license You apply to Your Copyright and Similar Rights in Your contributions to Adapted Material in accordance with the terms and conditions of this Public License.

c. **Copyright and Similar Rights** means copyright and/or similar rights closely related to copyright including, without limitation, performance, broadcast, sound recording, and Sui Generis Database Rights, without regard to how the rights are labeled or categorized. For purposes of this Public License, the rights specified in Section 2(b)(1)-(2) are not Copyright and Similar Rights.

d. **Effective Technological Measures** means those measures that, in the absence of proper authority, may not be circumvented under laws fulfilling obligations under Article 11 of the WIPO Copyright Treaty adopted on December 20, 1996, and/or similar international agreements.

e. **Exceptions and Limitations** means fair use, fair dealing, and/or any other exception or limitation to Copyright and Similar Rights that applies to Your use of the Licensed Material.

f. **Licensed Material** means the artistic or literary work, database, or other material to which the Licensor applied this Public License.

g. **Licensed Rights** means the rights granted to You subject to the terms and conditions of this Public License, which are limited to all Copyright and Similar Rights that apply to Your use of the Licensed Material and that the Licensor has authority to license.

h. **Licensor** means the individual(s) or entity(ies) granting rights under this Public License.

i. **Share** means to provide material to the public by any means or process that requires permission under the Licensed Rights, such as reproduction, public display, public performance, distribution, dissemination, communication, or importation, and to make material available to the public including in ways that members of the public may access the material from a place and at a time individually chosen by them.

j. **Sui Generis Database Rights** means rights other than copyright resulting from Directive 96/9/EC of the European Parliament and of the Council of 11 March 1996 on the legal protection of databases, as amended and/or succeeded, as well as other essentially equivalent rights anywhere in the world.

k. **You** means the individual or entity exercising the Licensed Rights under this Public License. **Your** has a corresponding meaning.

C.1.2 Section 2 – Scope.

a. ***License grant.***

   a. Subject to the terms and conditions of this Public License, the Licensor hereby grants You a worldwide, royalty-free, non-sublicensable, non-exclusive, irrevocable license to exercise the Licensed Rights in the Licensed Material to:

      A. reproduce and Share the Licensed Material, in whole or in part; and

      B. produce, reproduce, and Share Adapted Material.

   b. **Exceptions and Limitations.** For the avoidance of doubt, where Exceptions and Limitations apply to Your use, this Public License does not apply, and You do not need to comply with its terms and conditions.

   c. **Term.** The term of this Public License is specified in Section 6(a).

   d. **Media and formats; technical modifications allowed.** The Licensor authorizes You to exercise the Licensed Rights in all media and formats whether now known or hereafter created, and to make technical modifications necessary to do so. The Licensor waives and/or agrees not to assert any right or authority to forbid You from making technical modifications necessary to exercise the Licensed Rights, including technical modifications necessary to circumvent Effective Technological Measures. For purposes of this Public License, simply making modifications authorized by this Section 2(a)(4) never produces Adapted Material.

   e. **Downstream recipients.**

      A. **Offer from the Licensor – Licensed Material.** Every recipient of the Licensed Material automatically receives an offer from the Licensor to exercise the Licensed Rights under the terms and conditions of this Public License.

      B. **No downstream restrictions.** You may not offer or impose any additional or different terms or conditions on, or apply any Effective Technological Measures to, the Licensed Material if doing so restricts exercise of the Licensed Rights by any recipient of the Licensed Material.

f. **No endorsement.** Nothing in this Public License constitutes or may be construed as permission to assert or imply that You are, or that Your use of the Licensed Material is, connected with, or sponsored, endorsed, or granted official status by, the Licensor or others designated to receive attribution as provided in Section 3(a)(1)(A)(i).

b. ***Other rights.***

a. Moral rights, such as the right of integrity, are not licensed under this Public License, nor are publicity, privacy, and/or other similar personality rights; however, to the extent possible, the Licensor waives and/or agrees not to assert any such rights held by the Licensor to the limited extent necessary to allow You to exercise the Licensed Rights, but not otherwise.

b. Patent and trademark rights are not licensed under this Public License.

c. To the extent possible, the Licensor waives any right to collect royalties from You for the exercise of the Licensed Rights, whether directly or through a collecting society under any voluntary or waivable statutory or compulsory licensing scheme. In all other cases the Licensor expressly reserves any right to collect such royalties.

C.1.3 Section 3 – License Conditions.

Your exercise of the Licensed Rights is expressly made subject to the following conditions.

a. ***Attribution.***

a. If You Share the Licensed Material (including in modified form), You must:

A. retain the following if it is supplied by the Licensor with the Licensed Material:

i. identification of the creator(s) of the Licensed Material and any others designated to receive attribution, in any reasonable manner requested by the Licensor (including by pseudonym if designated);

ii. a copyright notice;

iii. a notice that refers to this Public License;

iv. a notice that refers to the disclaimer of warranties;

v. a URI or hyperlink to the Licensed Material to the extent reasonably practicable;

B. indicate if You modified the Licensed Material and retain an indication of any previous modifications; and

C. indicate the Licensed Material is licensed under this Public License, and include the text of, or the URI or hyperlink to, this Public License.

b. You may satisfy the conditions in Section 3(a)(1) in any reasonable manner based on the medium, means, and context in which You Share the Licensed Material. For example, it may be reasonable to satisfy the conditions by providing a URI or hyperlink to a resource that includes the required information.

c. If requested by the Licensor, You must remove any of the information required by Section 3(a)(1)(A) to the extent reasonably practicable.

d. If You Share Adapted Material You produce, the Adapter's License You apply must not prevent recipients of the Adapted Material from complying with this Public License.

C.1.4 Section 4 – Sui Generis Database Rights.

Where the Licensed Rights include Sui Generis Database Rights that apply to Your use of the Licensed Material:

a. for the avoidance of doubt, Section 2(a)(1) grants You the right to extract, reuse, reproduce, and Share all or a substantial portion of the contents of the database;

b. if You include all or a substantial portion of the database contents in a database in which You have Sui Generis Database Rights, then the database in which You have Sui Generis Database Rights (but not its individual contents) is Adapted Material; and

c. You must comply with the conditions in Section 3(a) if You Share all or a substantial portion of the contents of the database.

For the avoidance of doubt, this Section 4 supplements and does not replace Your obligations under this Public License where the Licensed Rights include other Copyright and Similar Rights.

C.1.5 Section 5 – Disclaimer of Warranties and Limitation of Liability.

a. **Unless otherwise separately undertaken by the Licensor, to the extent possible, the Licensor offers the Licensed Material as-is and as-available, and makes no representations or warranties of any kind concerning the Licensed Material, whether express, implied, statutory, or other. This includes, without limitation, warranties of title, merchantability, fitness for a particular purpose, non-infringement, absence of latent or other defects, accuracy, or the presence or absence of errors, whether or not known or discoverable. Where disclaimers of warranties are not allowed in full or in part, this disclaimer may not apply to You.**

b. **To the extent possible, in no event will the Licensor be liable to You on any legal theory (including, without limitation, negligence) or otherwise for any direct, special, indirect, incidental, consequential, punitive, exemplary, or other losses, costs, expenses, or damages arising out of this Public License or use of the Licensed Material, even if the Licensor has been advised of the possibility of such losses, costs, expenses, or damages. Where a limitation of liability is not allowed in full or in part, this limitation may not apply to You.**

c. The disclaimer of warranties and limitation of liability provided above shall be interpreted in a manner that, to the extent possible, most closely approximates an absolute disclaimer and waiver of all liability.

C.1.6 Section 6 – Term and Termination.

a. This Public License applies for the term of the Copyright and Similar Rights licensed here. However, if You fail to comply with this Public License, then Your rights under this Public License terminate automatically.

b. Where Your right to use the Licensed Material has terminated under Section 6(a), it reinstates:

   a. automatically as of the date the violation is cured, provided it is cured within 30 days of Your discovery of the violation; or

   b. upon express reinstatement by the Licensor.

   For the avoidance of doubt, this Section 6(b) does not affect any right the Licensor may have to seek remedies for Your violations of this Public License.

c. For the avoidance of doubt, the Licensor may also offer the Licensed Material under separate terms or conditions or stop distributing the Licensed Material at any time; however, doing so will not terminate this Public License.

d. Sections 1, 5, 6, 7, and 8 survive termination of this Public License.

C.1.7 Section 7 – Other Terms and Conditions.

a. The Licensor shall not be bound by any additional or different terms or conditions communicated by You unless expressly agreed.

b. Any arrangements, understandings, or agreements regarding the Licensed Material not stated herein are separate from and independent of the terms and conditions of this Public License.

C.1.8 Section 8 – Interpretation.

a. For the avoidance of doubt, this Public License does not, and shall not be interpreted to, reduce, limit, restrict, or impose conditions on any use of the Licensed Material that could lawfully be made without permission under this Public License.

b. To the extent possible, if any provision of this Public License is deemed unenforceable, it shall be automatically reformed to the minimum extent necessary to make it enforceable. If the provision cannot be reformed, it shall be severed from this Public License without affecting the enforceability of the remaining terms and conditions.

c. No term or condition of this Public License will be waived and no failure to comply consented to unless expressly agreed to by the Licensor.

d. Nothing in this Public License constitutes or may be interpreted as a limitation upon, or waiver of, any privileges and immunities that apply to the Licensor or You, including from the legal processes of any jurisdiction or authority.

Creative Commons is not a party to its public licenses. Notwithstanding, Creative Commons may elect to apply one of its public licenses to material it publishes and in those instances will be considered the "Licensor." Except for the limited purpose of indicating that material is shared under a Creative Commons public license or as otherwise permitted by the Creative Commons policies published at creativecommons.org/policies, Creative Commons does not authorize the use of the trademark "Creative Commons" or any other trademark or logo of Creative Commons without its prior written consent including, without limitation, in connection with any unauthorized modifications to any of its public licenses or any other arrangements, understandings, or agreements concerning use of licensed material. For the avoidance of doubt, this paragraph does not form part of the public licenses.

Creative Commons may be contacted at creativecommons.org